PRACTICAL FILE

Advanced Java Programming

(Paper 1T1)

First Year M.C.A. — Semester I (CBCS)

RTM Nagpur University

Name: _____

Roll No: _____

INDEX

Sr. No.	Practical	Date	Sign
1	Write a java application to implement interfaces.		
2	Write a java application to implement final class.		
3	Write a java application to implement Method Overloading.		
4	Write a java application to implement multithreading.		
5	Write a java application to implement Single level inheritance.		
6	Write a java application to implement passing parameter to Applet.		
7	Write an AWT based java application for Text Area control.		
8	Write an AWT based java application to implement Item Event of Check Box control.		
9	Write a SWING based java application to implement Click event of Button control.		
10	Write an AWT based java application for Menus.		
11	Write a SWING based java application to implement different Layout Manager.		
12	Write a java application to create server and client program using TCP/IP protocol.		
13	Write a java application to create server and client program using Datagram.		
14	Write a java application to implement RMI program for addition of two numbers.		
15	Write a java application to insert data into database using JDBC.		

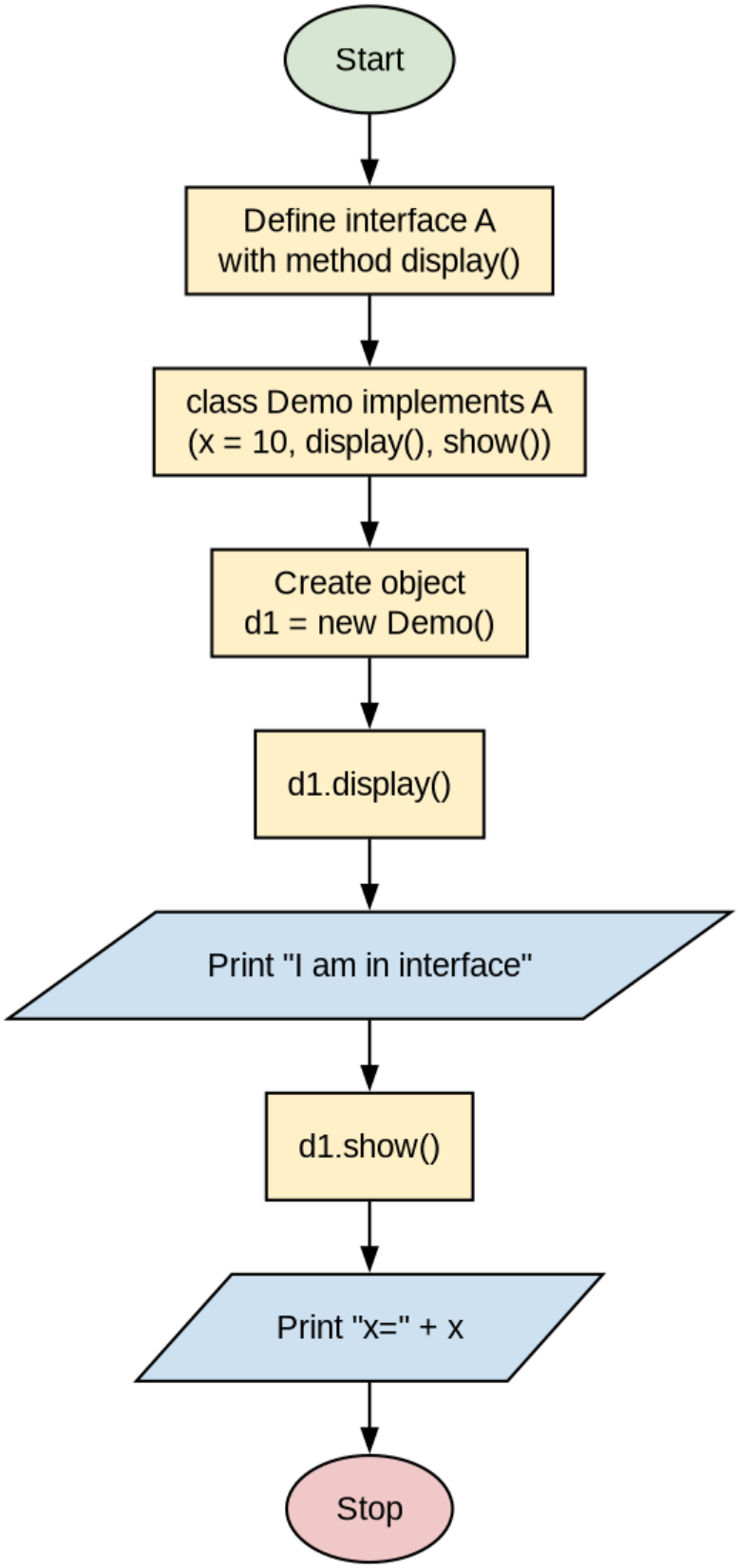
Practical No. 1

Aim: Write a java application to implement interfaces.

Algorithm:

Step 1: Start.
Step 2: Define an interface A with an abstract method display().
Step 3: Define class Demo which implements interface A and has a variable x = 10.
Step 4: Override display() in class Demo to print "I am in interface".
Step 5: Define method show() to print the value of x.
Step 6: In main(), create object d1 of class Demo.
Step 7: Call d1.display() and d1.show().
Step 8: Stop.

Flowchart:


Start
Define interface A
with method display()
class Demo implements A
(x = 10, display(), show())
Create object
d1 = new Demo()
d1.display()
Print "I am in interface"
d1.show()
Print "x=" + x
Stop

Program:

```
interface A
{
    void display();
}

public class Demo implements A
{
    int x = 10;

    public void display()
    {
        System.out.println("I am in interface");
    }

    public void show()
    {
        System.out.println("x=" + x);
    }

    public static void main(String[] args)
    {
        Demo d1 = new Demo();
        d1.display();
        d1.show();
    }
}
```

Output:

```
I am in interface
x=10
```

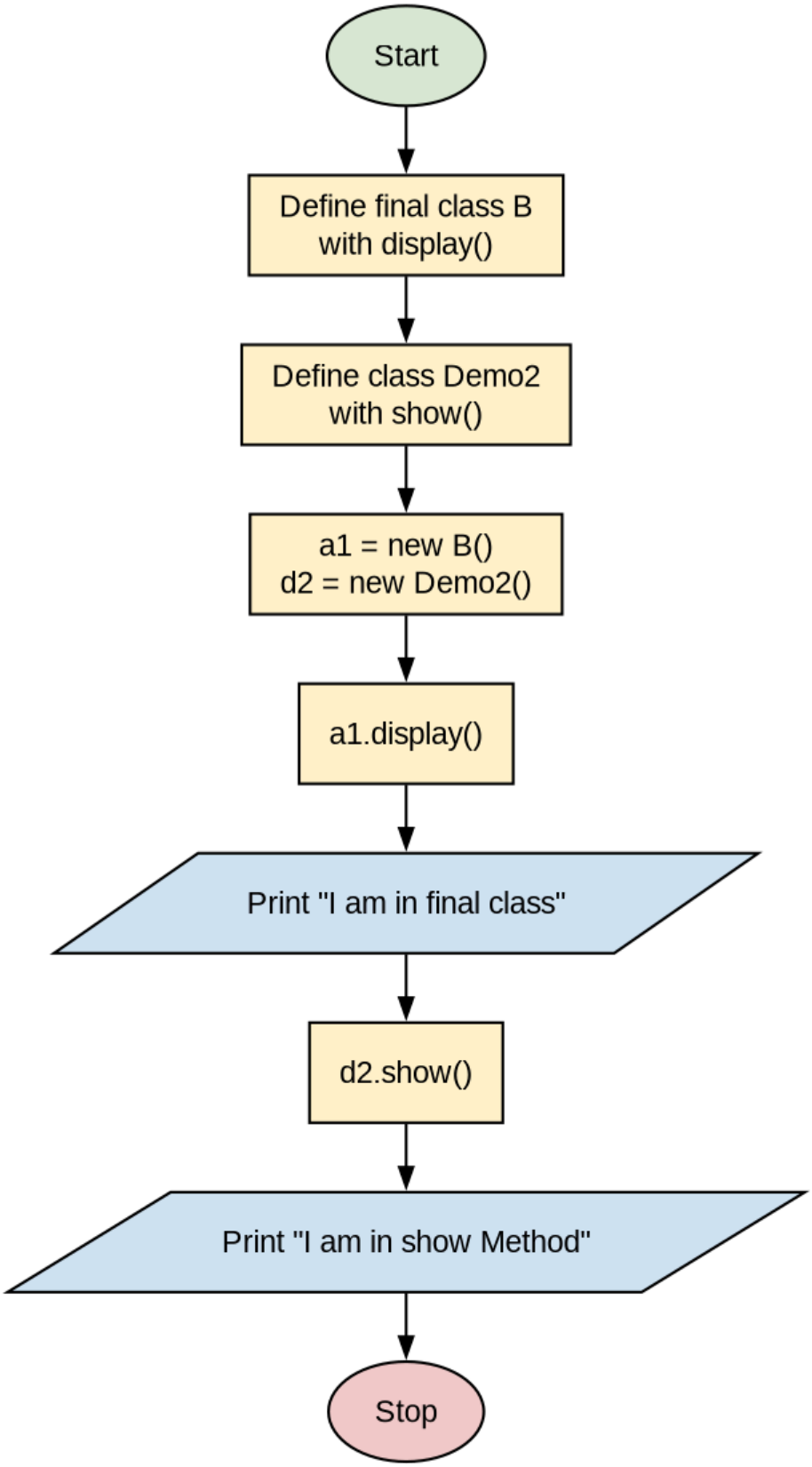
Practical No. 2

Aim: Write a java application to implement final class.

Algorithm:

Step 1: Start.
Step 2: Define a final class B with method display() (a final class cannot be inherited).
Step 3: Define class Demo2 with method show().
Step 4: In main(), create object a1 of class B and object d2 of class Demo2.
Step 5: Call a1.display() and d2.show().
Step 6: Stop.

Flowchart:


Start
Define final class B with display()
Define class Demo2 with show()
a1 = new B()
d2 = new Demo2()
a1.display()
Print "I am in final class"
d2.show()
Print "I am in show Method"
Stop

Program:

```
final class B
{
    void display()
    {
        System.out.println("I am in final class");
    }
}

public class Demo2
{
    public void show()
    {
        System.out.println("I am in show Method");
    }

    public static void main(String[] args)
    {
        B a1 = new B();
        Demo2 d2 = new Demo2();
        a1.display();
        d2.show();
    }
}
```

Output:

```
I am in final class
I am in show Method
```

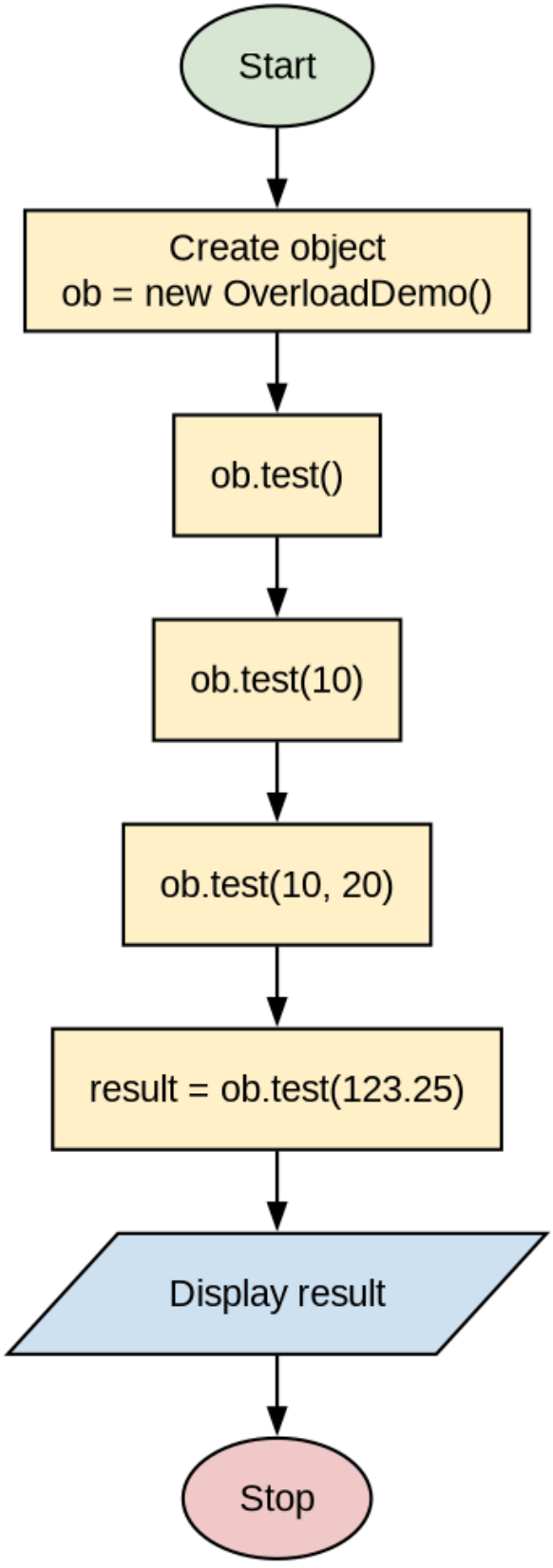
Practical No. 3

Aim: Write a java application to implement Method Overloading.

Algorithm:

Step 1: Start.
Step 2: Define class OverloadDemo with four methods named test():
    test() with no parameters.
    test(int a) with one integer parameter.
    test(int a, int b) with two integer parameters.
    test(double a) which returns a * a.
Step 3: In main(), create object ob of class OverloadDemo.
Step 4: Call ob.test(), ob.test(10), ob.test(10, 20) and result = ob.test(123.25).
Step 5: Display the result.
Step 6: Stop.

Flowchart:


Start
Create object
ob = new OverloadDemo()
ob.test()
ob.test(10)
ob.test(10, 20)
result = ob.test(123.25)
Display result
Stop

Program:

```
class OverloadDemo {
    // Overload test with no parameters
    void test() {
        System.out.println("No parameters");
    }

    // Overload test for one integer parameter
    void test(int a) {
        System.out.println("a: " + a);
    }

    // Overload test for two integer parameters
    void test(int a, int b) {
        System.out.println("a and b: " + a + " " + b);
    }

    // Overload test for a double parameter
    double test(double a) {
        System.out.println("double a: " + a);
        return a * a;
    }
}

class Overload {
    public static void main(String args[]) {
        OverloadDemo ob = new OverloadDemo();
        double result;

        // Call all versions of test()
        ob.test();                // no-arg version
        ob.test(10);              // one int argument
        ob.test(10, 20);          // two int arguments
        result = ob.test(123.25);  // double argument

        System.out.println("Result of ob.test(123.25): " + result);
    }
}
```

Output:

```
No parameters
a: 10
a and b: 10 20
double a: 123.25
Result of ob.test(123.25): 15190.5625
```

Practical No. 4

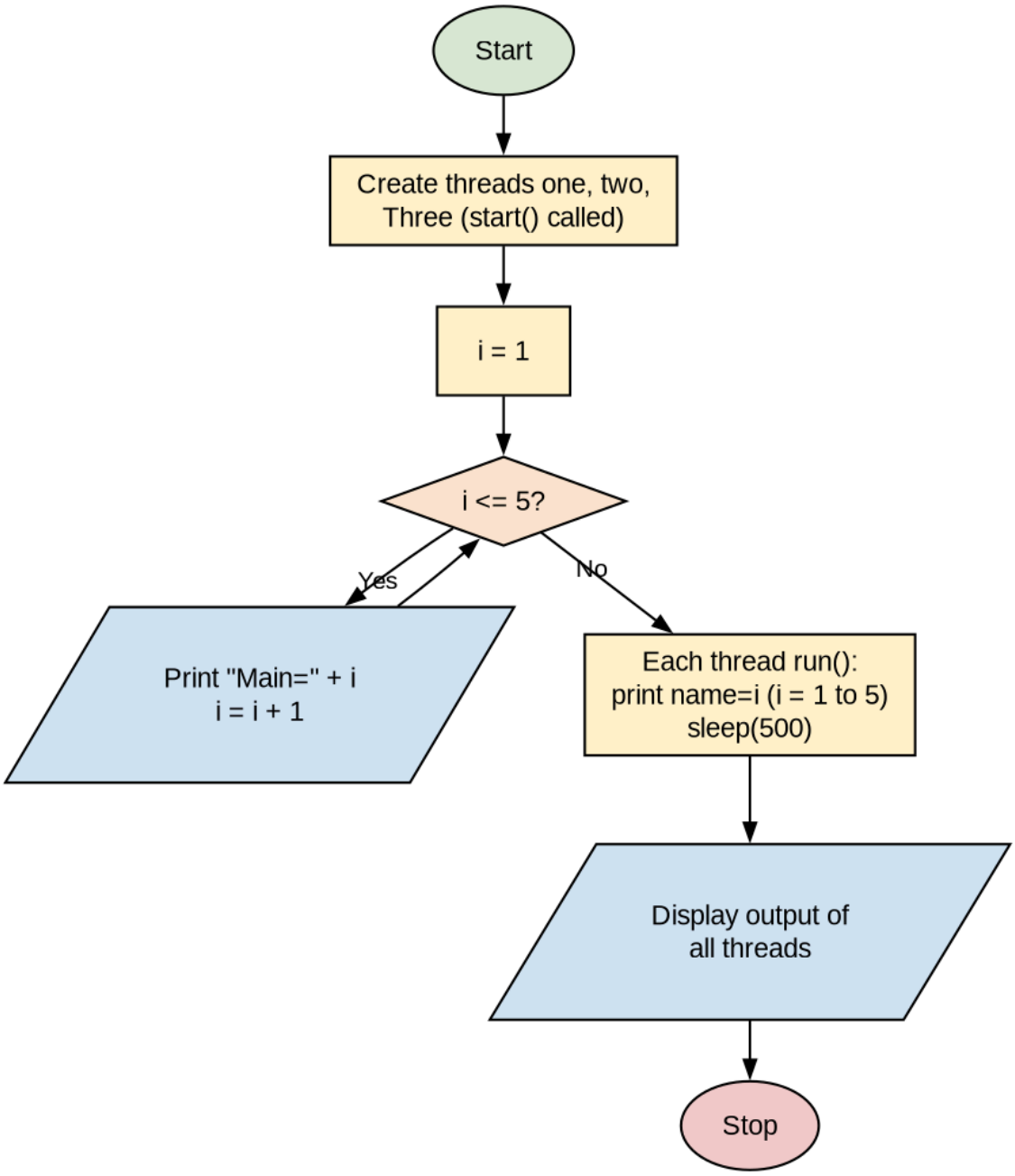
Aim: Write a java application to implement multithreading.

Algorithm:

Step 1: Start.
Step 2: Define class NewThread which extends Thread.
Step 3: In the constructor, set the thread name using super(n) and call start().
Step 4: In run(), print the thread name and i for i = 1 to 5, sleeping 500 ms after each print.
Step 5: In main(), create three threads named one, two and Three.
Step 6: In the main thread, print Main=i for i = 1 to 5.
Step 7: All threads run concurrently and print their values.
Step 8: Stop.

Note: *The output is corrected here. Thread names print exactly as given in the program (one, two, Three), and the order of thread output may change on every run. Please test and run it yourself before writing.*

Flowchart:


Start
Create threads one, two,
Three (start() called)
i = 1
i <= 5?
Yes
No
Print "Main=" + i
i = i + 1
Each thread run():
print name=i (i = 1 to 5)
sleep(500)
Display output of
all threads
Stop

Program:

```
class NewThread extends Thread
{
    public NewThread(String n)
    {
        super(n);
        start();
    }

    public void run()
    {
        for (int i = 1; i <= 5; i++)
        {
            System.out.println(getName() + "=" + i);
            try
            {
                Thread.sleep(500);
            }
            catch (Exception e)
            {
                System.out.println(e);
            }
        }
    }
}

public class ThreadDemo
{
    public static void main(String[] args)
    {
        NewThread n1 = new NewThread("one");
        NewThread n2 = new NewThread("two");
        NewThread n3 = new NewThread("Three");
        for (int i = 1; i <= 5; i++)
            System.out.println("Main=" + i);
        try
        {
            Thread.sleep(500);
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

```
Main=1
Main=2
Main=3
Main=4
Main=5
one=1
two=1
Three=1
one=2
two=2
Three=2
one=3
two=3
Three=3
one=4
two=4
Three=4
one=5
two=5
Three=5
```

Practical No. 5

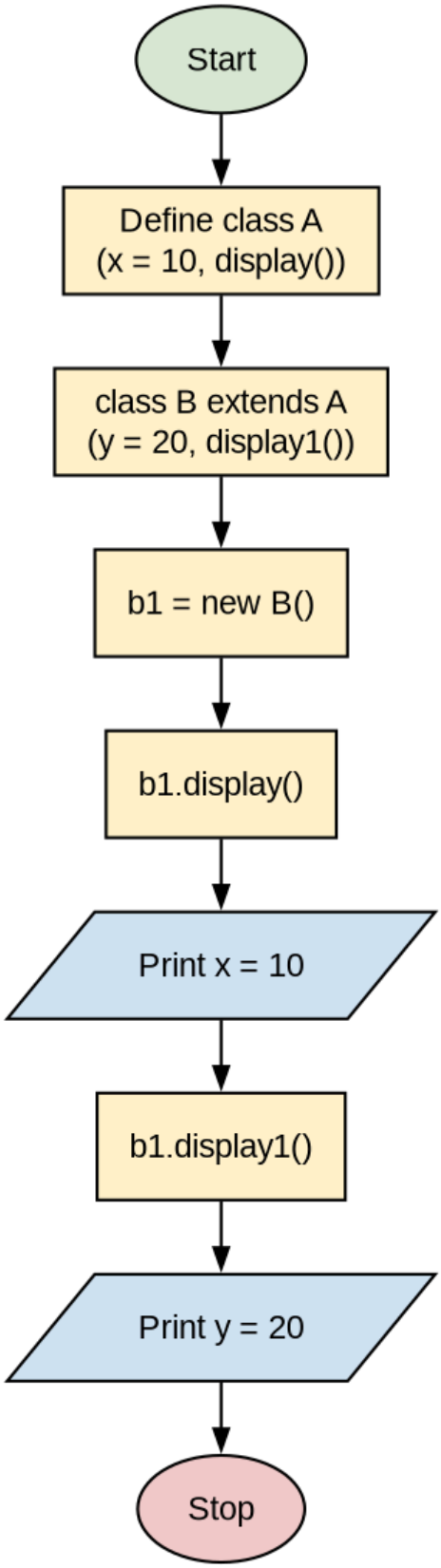
Aim: Write a java application to implement Single level inheritance.

Algorithm:

Step 1: Start.
Step 2: Define base class A with variable x = 10 and method display().
Step 3: Define class B which extends A, with variable y = 20 and method display1().
Step 4: In main(), create object b1 of subclass B.
Step 5: Call b1.display() (inherited from A) and b1.display1() (of B).
Step 6: Stop.

Note: *The output is corrected here: the program prints "x = 10" and "y = 20" with spaces. Please test and run it yourself before writing.*

Flowchart:


Start
Define class A
(x = 10, display())
class B extends A
(y = 20, display1())
b1 = new B()
b1.display()
Print x = 10
b1.display1()
Print y = 20
Stop

Program:

```
class A {
    int x = 10;

    public void display() {
        System.out.println("x = " + x);
    }
}

class B extends A {
    int y = 20;

    public void display1() {
        System.out.println("y = " + y);
    }
}

public class InheritDemo {
    public static void main(String args[]) {
        B b1 = new B();   // Object of subclass B
        b1.display();     // Calls method of class A
        b1.display1();    // Calls method of class B
    }
}
```

Output:

```
x = 10
y = 20
```

Practical No. 6

Aim: Write a java application to implement passing parameter to Applet.

Algorithm:

Step 1: Start.

Step 2: Create an HTML file with the <applet> tag and pass a parameter using <param name="string" value="Alex">.

Step 3: Create class ParamDemo which extends Applet.

Step 4: In init(), read the parameter using getParameter("string").

Step 5: If the parameter is null, set str = "java"; otherwise set str = "Hello " + str.

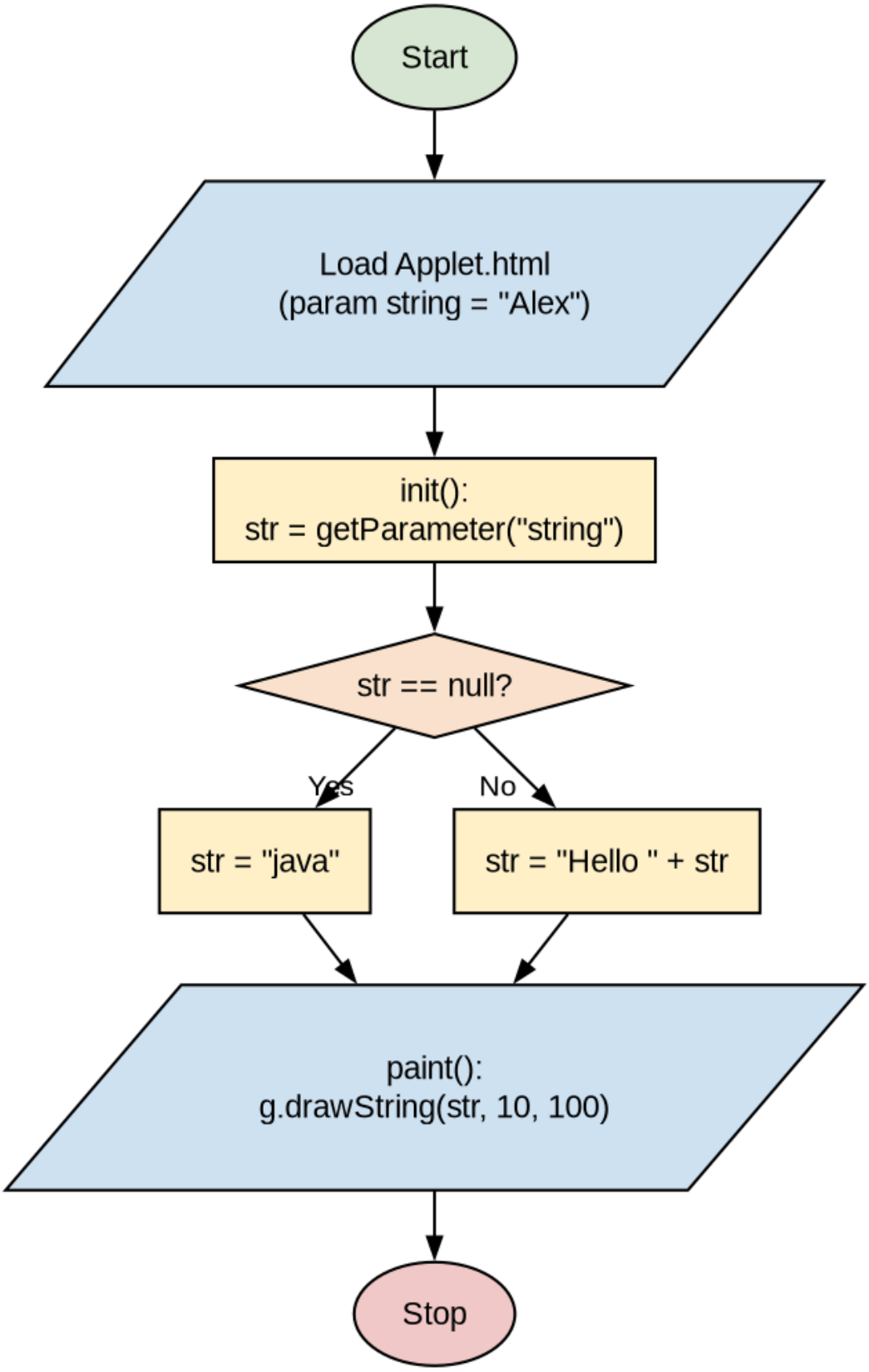
Step 6: In paint(), display str using g.drawString().

Step 7: Compile ParamDemo.java and run Applet.html using appletviewer.

Step 8: Stop.

Note: *The answer is corrected here. In the HTML file, code should be "ParamDemo.class" (not .java), and the closing quote after "Alex" was missing. A space is also added after "Hello". Run it with appletviewer (JDK 8), because browsers no longer run applets. Please test and run it yourself before writing.*

Flowchart:


Start
Load Applet.html
(param string = "Alex")
init():
str = getParameter("string")
str == null?
Yes
No
str = "java"
str = "Hello " + str
paint():
g.drawString(str, 10, 100)
Stop

Program:

```
File: Applet.html
-----------------
<html>
<body>
<applet code="ParamDemo.class" width="400" height="400">
<param name="string" value="Alex">
</applet>
</body>
</html>

File: ParamDemo.java
--------------------
import java.awt.*;
import java.applet.*;

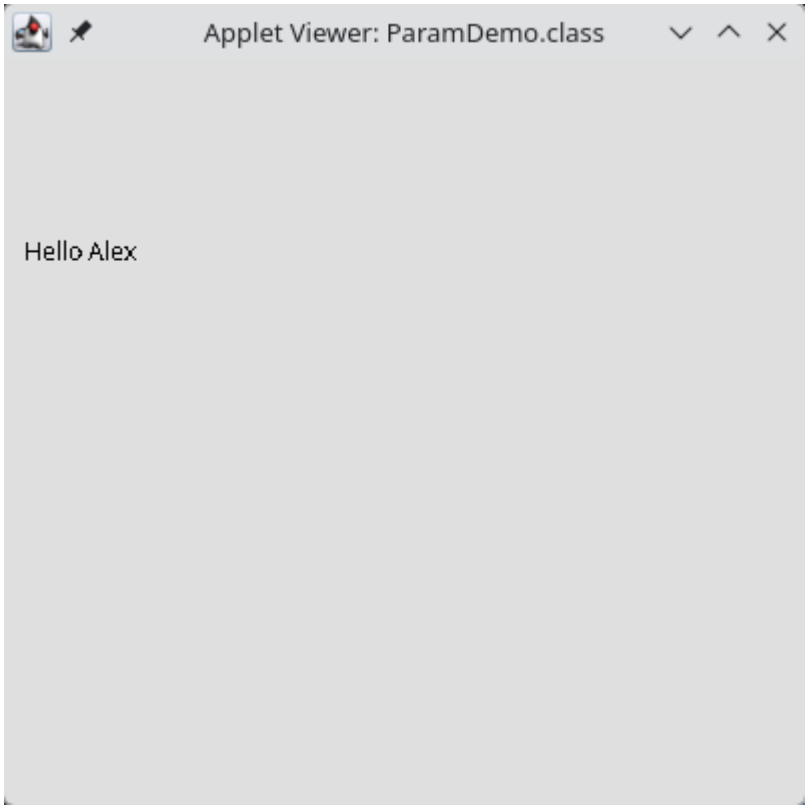
public class ParamDemo extends Applet
{
    String str;

    public void init()
    {
        str = getParameter("string");
        if (str == null)
            str = "java";
        else
            str = "Hello " + str;
    }

    public void paint(Graphics g)
    {
        g.drawString(str, 10, 100);
    }
}
```

Output:

```
C:\>javac ParamDemo.java
C:\>appletviewer Applet.html
```


Applet Viewer: ParamDemo.class
Hello Alex

Practical No. 7

Aim: Write an AWT based java application for Text Area control.

Algorithm:

Step 1: Start.

Step 2: Create class TextAreaApp which extends Frame.

Step 3: Create two TextAreas t1 and t2 and a Button "copy", and place them using setBounds() (null layout).

Step 4: Register an ActionListener on the button; on click, copy the text of t1 into t2 using t2.setText(t1.getText()).

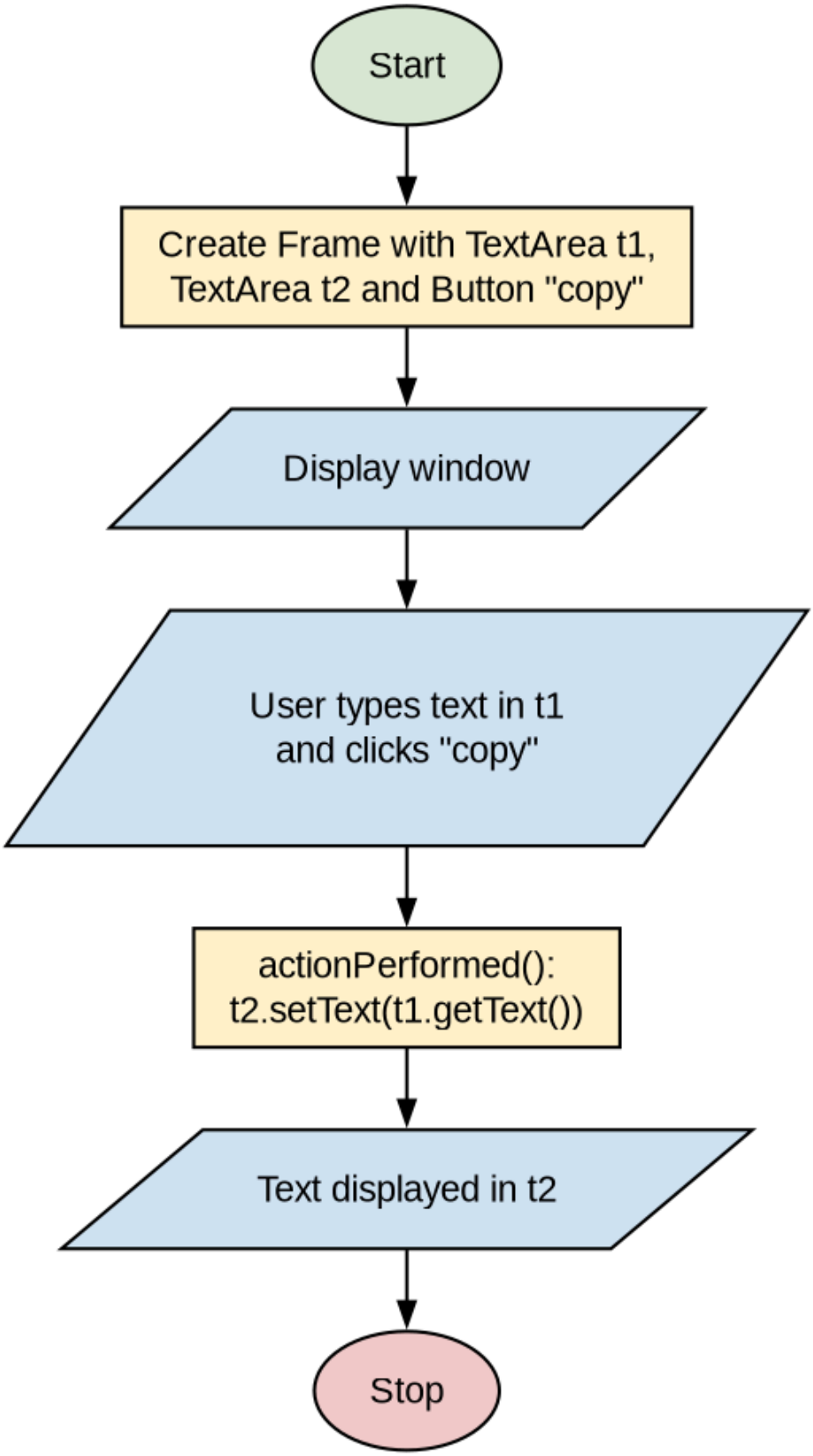
Step 5: Add a WindowListener to close the window.

Step 6: In main(), create the frame and make it visible.

Step 7: Stop.

Note: *The output was not given in the answer sheet; it is added here. Please test and run it yourself before writing.*

Flowchart:


Start
Create Frame with TextArea t1, TextArea t2 and Button "copy"
Display window
User types text in t1 and clicks "copy"
actionPerformed(): t2.setText(t1.getText())
Text displayed in t2
Stop

Program:

```
import java.awt.*;
import java.awt.event.*;

public class TextAreaApp extends Frame {

    TextArea t1, t2;
    Button b1;

    public TextAreaApp() {
        setTitle("my window");
        setSize(400, 400);
        setLayout(null);

        t1 = new TextArea();
        t1.setBounds(20, 50, 200, 150);
        add(t1);

        t2 = new TextArea();
        t2.setBounds(20, 245, 200, 150);
        add(t2);

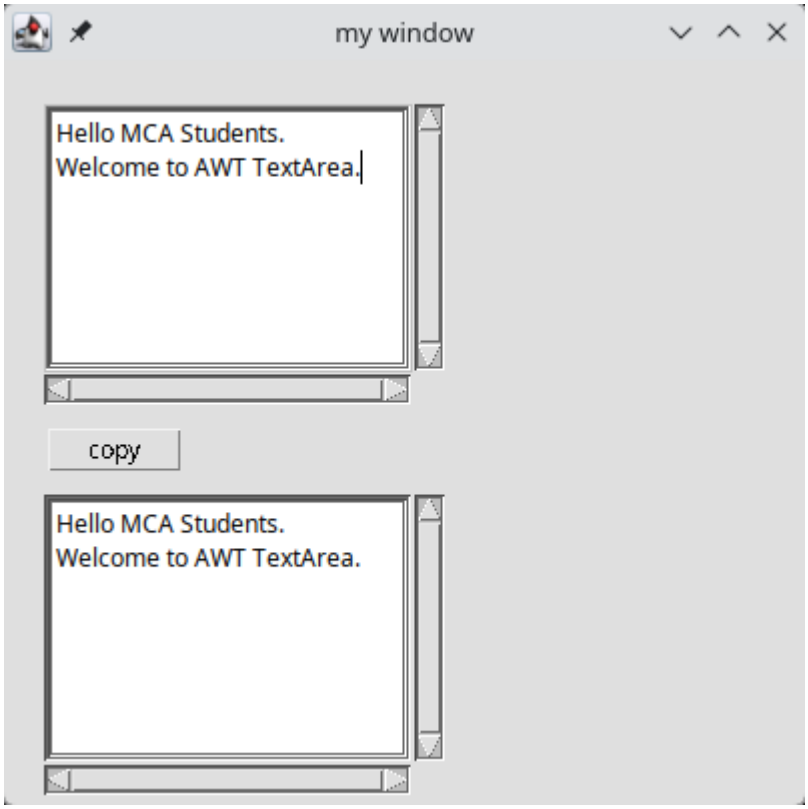
        b1 = new Button("copy");
        b1.setBounds(20, 210, 70, 25);
        add(b1);

        b1.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                t2.setText(t1.getText());
            }
        });

        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });
    }

    public static void main(String[] args) {
        TextAreaApp f = new TextAreaApp();
        f.setVisible(true);
    }
}
```

Output:


my window
Hello MCA Students.
Welcome to AWT TextArea.
copy
Hello MCA Students.
Welcome to AWT TextArea.

Practical No. 8

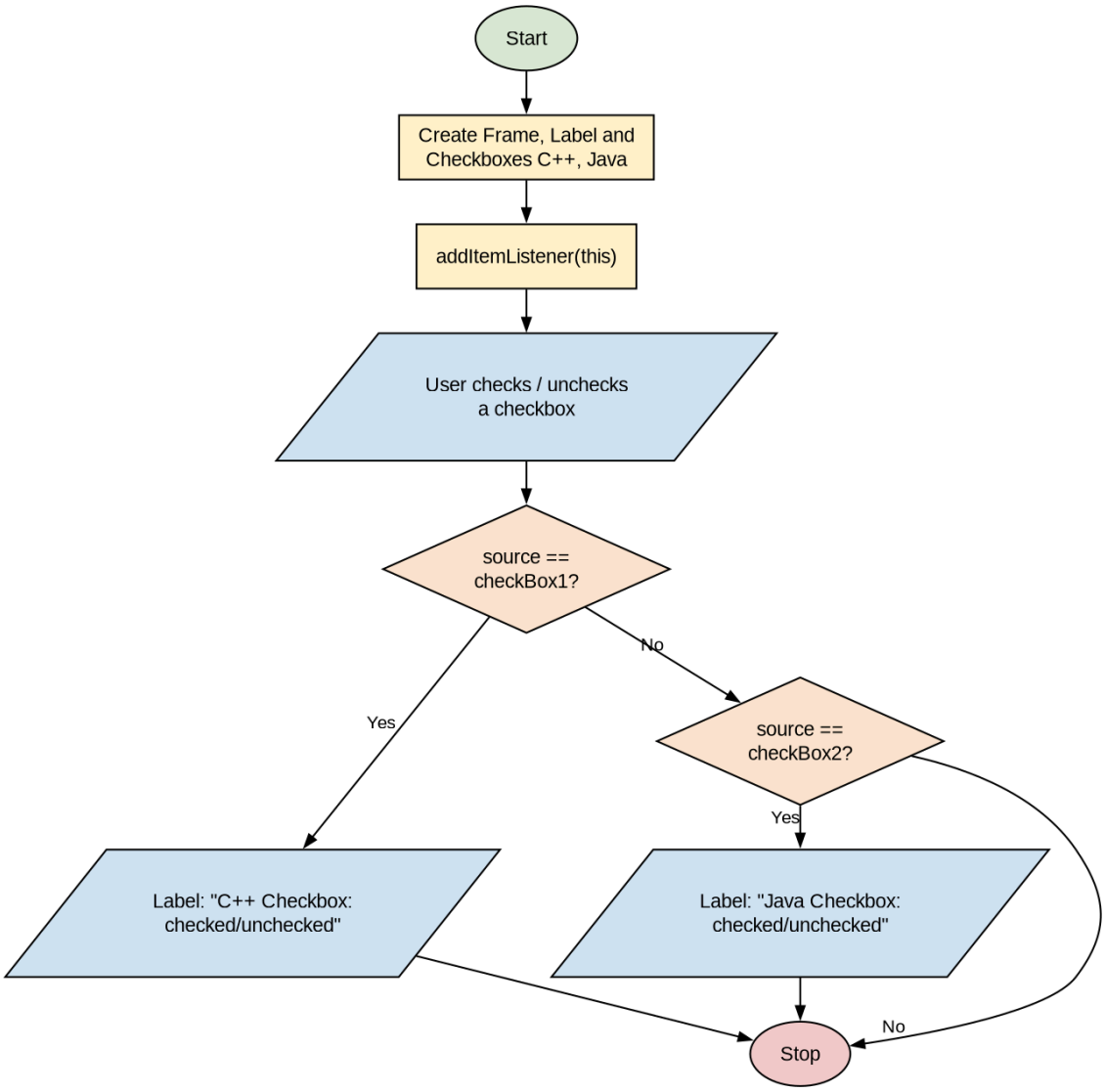
Aim: Write an AWT based java application to implement Item Event of Check Box control.

Algorithm:

Step 1: Start.
Step 2: Create class MyDemo which implements ItemListener.
Step 3: Create a Frame, a Label and two Checkboxes "C++" and "Java".
Step 4: Register the ItemListener on both checkboxes using addItemListener(this).
Step 5: In itemStateChanged(), check which checkbox caused the event using e.getSource().
Step 6: Display "checked" or "unchecked" in the label using e.getStateChange().
Step 7: Stop.

Note: *The output was not given in the answer sheet; it is added here. Please test and run it yourself before writing.*

Flowchart:


Start
Create Frame, Label and Checkboxes C++, Java
addItemListener(this)
User checks / unchecks a checkbox
source == checkBox1?
Yes
No
source == checkBox2?
Yes
No
Label: "C++ Checkbox: checked/unchecked"
Label: "Java Checkbox: checked/unchecked"
Stop

Program:

```
import java.awt.*;
import java.awt.event.*;

public class MyDemo implements ItemListener
{
    Checkbox checkBox1, checkBox2;
    Label label;

    MyDemo()
    {
        Frame f = new Frame("CheckBox Example");
        label = new Label();
        label.setAlignment(Label.CENTER);
        label.setSize(400, 100);

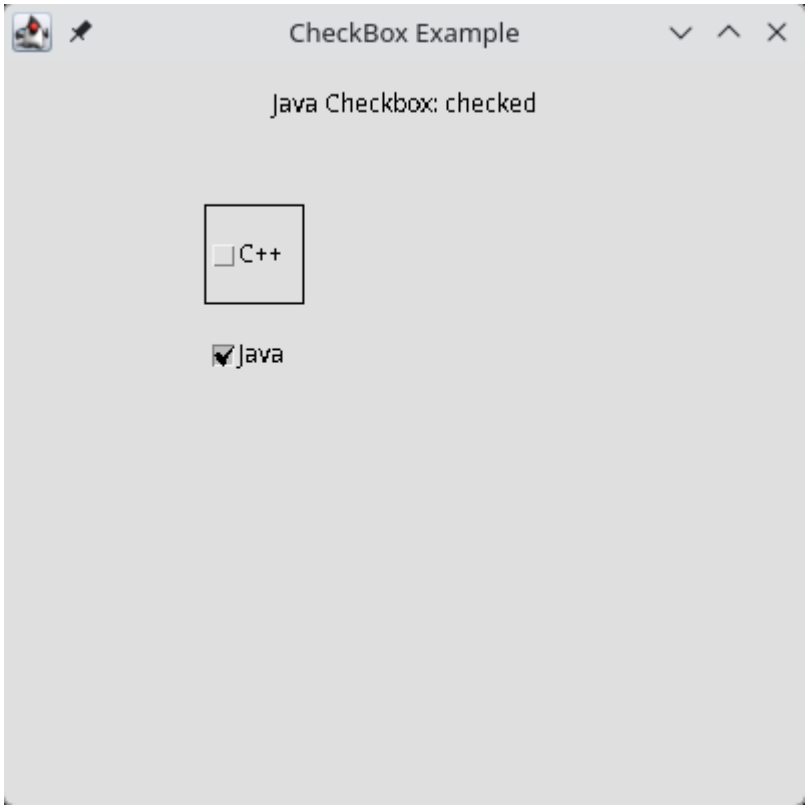
        checkBox1 = new Checkbox("C++");
        checkBox1.setBounds(100, 100, 50, 50);
        checkBox2 = new Checkbox("Java");
        checkBox2.setBounds(100, 150, 50, 50);

        f.add(checkBox1);
        f.add(checkBox2);
        f.add(label);
        checkBox1.addItemListener(this);
        checkBox2.addItemListener(this);
        f.setSize(400, 400);
        f.setLayout(null);
        f.setVisible(true);
    }

    public void itemStateChanged(ItemEvent e)
    {
        if (e.getSource() == checkBox1)
            label.setText("C++ Checkbox: "
                    + (e.getStateChange() == 1 ? "checked" : "unchecked"));
        if (e.getSource() == checkBox2)
            label.setText("Java Checkbox: "
                    + (e.getStateChange() == 1 ? "checked" : "unchecked"));
    }

    public static void main(String[] args)
    {
        new MyDemo();
    }
}
```

Output:


CheckBox Example
Java Checkbox: checked
C++
Java

Practical No. 9

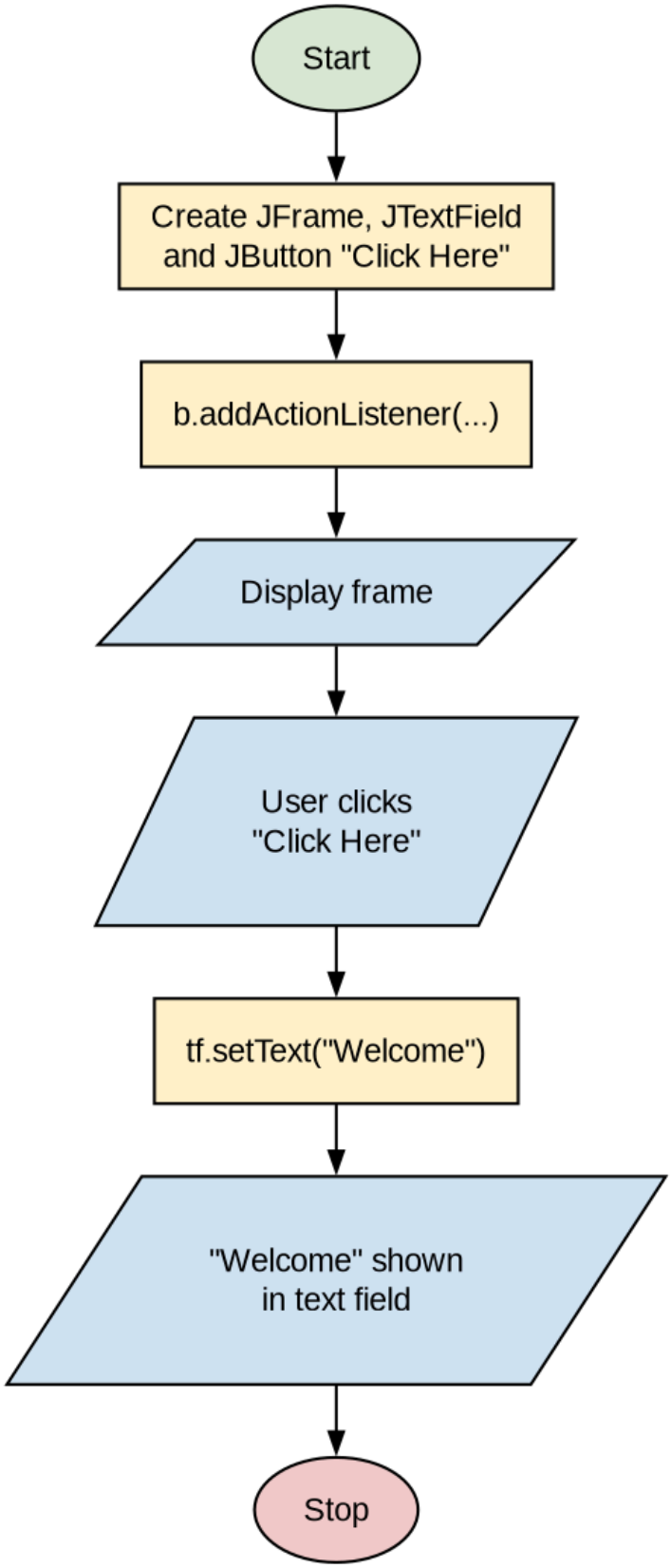
Aim: Write a SWING based java application to implement Click event of Button control.

Algorithm:

Step 1: Start.
Step 2: Create a JFrame, a JTextField and a JButton "Click Here".
Step 3: Set the positions using setBounds() with null layout.
Step 4: Register an ActionListener on the button.
Step 5: When the button is clicked, set the text field to "Welcome".
Step 6: Set the frame size and make it visible.
Step 7: Stop.

Note: *The output was not given in the answer sheet; it is added here. Please test and run it yourself before writing.*

Flowchart:


Start
Create JFrame, JTextField and JButton "Click Here"
b.addActionListener(...)
Display frame
User clicks "Click Here"
tf.setText("Welcome")
"Welcome" shown in text field
Stop

Program:

```
import java.awt.event.*;
import javax.swing.*;

public class ButtonExample {
    public static void main(String[] args) {

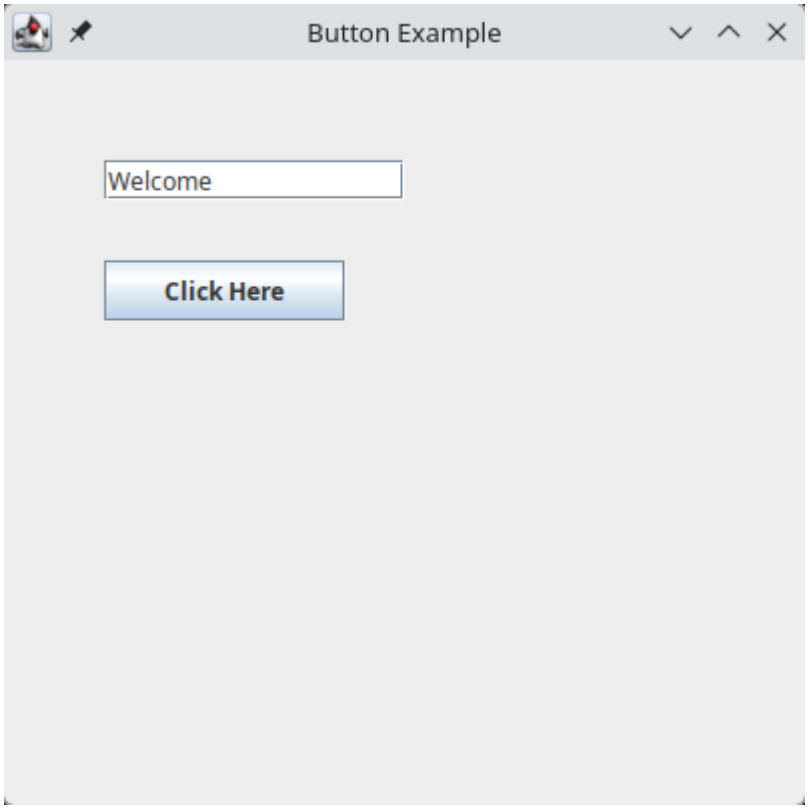
        JFrame f = new JFrame("Button Example");
        final JTextField tf = new JTextField();
        tf.setBounds(50, 50, 150, 20);

        JButton b = new JButton("Click Here");
        b.setBounds(50, 100, 120, 30);

        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                tf.setText("Welcome");
            }
        });

        f.add(b);
        f.add(tf);
        f.setSize(400, 400);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Output:



Practical No. 10

Aim: Write an AWT based java application for Menus.

Algorithm:

Step 1: Start.

Step 2: Create a Frame with GridLayout(3, 1) containing a header label, a panel and a status label.

Step 3: Create a MenuBar and menus File, Edit and About.

Step 4: Create menu items New, Open, Save, Exit, Cut, Copy and Paste, and register a MenuItemListener on each.

Step 5: Create a CheckboxMenuItem "Show About" which adds or removes the About menu.

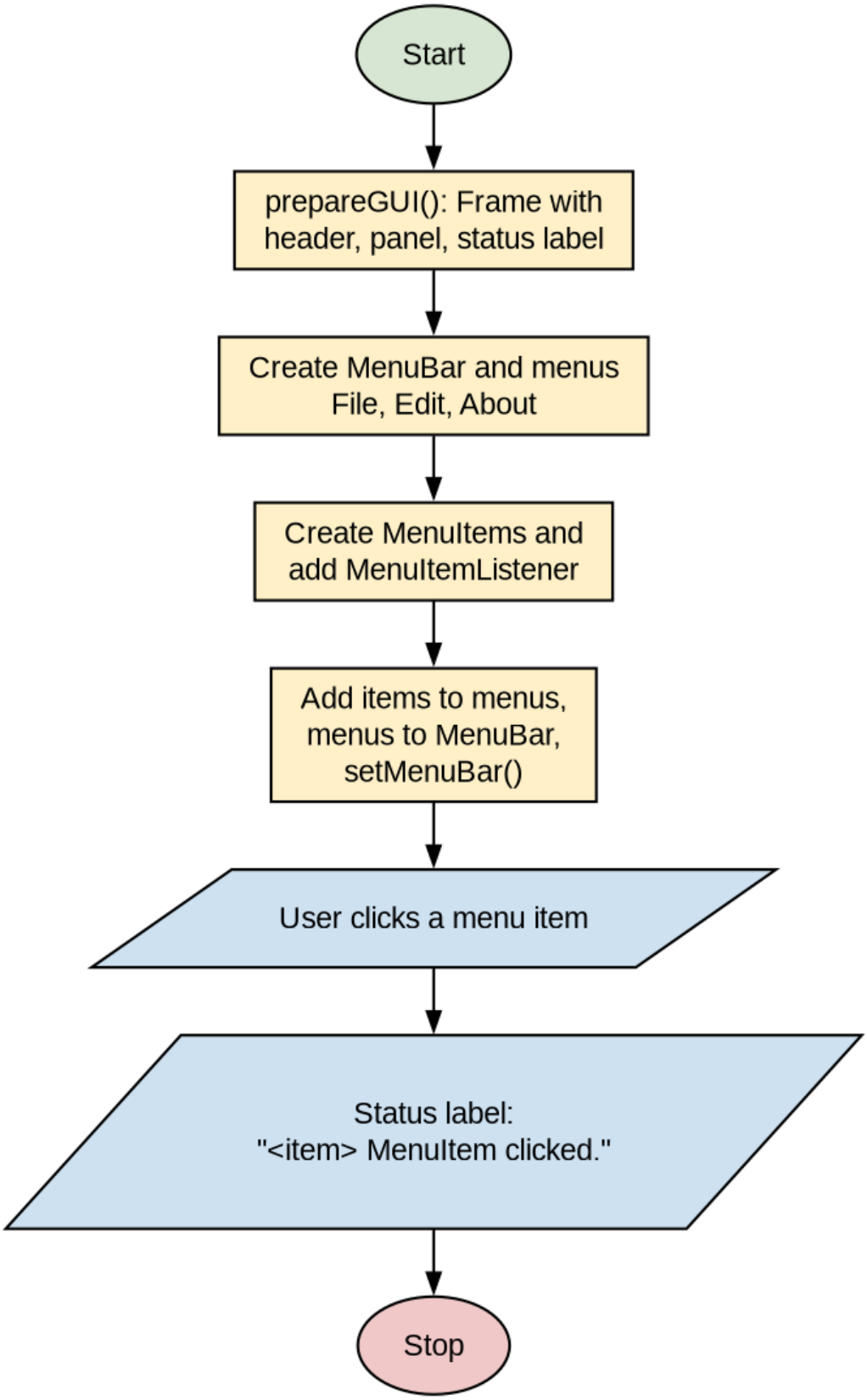
Step 6: Add the items to the menus, the menus to the MenuBar, and set the MenuBar on the frame.

Step 7: When a menu item is clicked, display "<item> MenuItem clicked." in the status label.

Step 8: Stop.

Note: *The output was not given in the answer sheet; it is added here (after clicking File > Open). Please test and run it yourself before writing.*

Flowchart:


Start
prepareGUI(): Frame with header, panel, status label
Create MenuBar and menus File, Edit, About
Create MenuItems and add MenuItemListener
Add items to menus, menus to MenuBar, setMenuBar()
User clicks a menu item
Status label: "<item> MenuItem clicked."
Stop

Program:

```
import java.awt.*;
import java.awt.event.*;

public class AWTMenuDemo {
    private Frame mainFrame;
    private Label headerLabel;
    private Label statusLabel;
    private Panel controlPanel;

    public AWTMenuDemo() {
        prepareGUI();
    }

    public static void main(String[] args) {
        AWTMenuDemo awtMenuDemo = new AWTMenuDemo();
        awtMenuDemo.showMenuDemo();
    }

    private void prepareGUI() {
        mainFrame = new Frame("Java AWT Example");
        mainFrame.setSize(400, 400);
        mainFrame.setLayout(new GridLayout(3, 1));

        mainFrame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent windowEvent) {
                System.exit(0);
            }
        });

        headerLabel = new Label();
        headerLabel.setAlignment(Label.CENTER);

        statusLabel = new Label();
        statusLabel.setAlignment(Label.CENTER);
        statusLabel.setSize(350, 100);

        controlPanel = new Panel();
        controlPanel.setLayout(new FlowLayout());

        mainFrame.add(headerLabel);
        mainFrame.add(controlPanel);
        mainFrame.add(statusLabel);
        mainFrame.setVisible(true);
    }

    private void showMenuDemo() {
        final MenuBar menuBar = new MenuBar();

        Menu fileMenu = new Menu("File");
        Menu editMenu = new Menu("Edit");
        final Menu aboutMenu = new Menu("About");

        MenuItem newMenuItem = new MenuItem("New", new MenuShortcut(KeyEvent.VK_N));
        newMenuItem.setActionCommand("New");

        MenuItem openMenuItem = new MenuItem("Open");
        openMenuItem.setActionCommand("Open");

        MenuItem saveMenuItem = new MenuItem("Save");
        saveMenuItem.setActionCommand("Save");

        MenuItem exitMenuItem = new MenuItem("Exit");
```

```
        exitMenuItem.setActionCommand("Exit");

        MenuItem cutMenuItem = new MenuItem("Cut");
        cutMenuItem.setActionCommand("Cut");

        MenuItem copyMenuItem = new MenuItem("Copy");
        copyMenuItem.setActionCommand("Copy");

        MenuItem pasteMenuItem = new MenuItem("Paste");
        pasteMenuItem.setActionCommand("Paste");

        MenuItemListener menuItemListener = new MenuItemListener();
        newMenuItem.addActionListener(menuItemListener);
        openMenuItem.addActionListener(menuItemListener);
        saveMenuItem.addActionListener(menuItemListener);
        exitMenuItem.addActionListener(menuItemListener);
        cutMenuItem.addActionListener(menuItemListener);
        copyMenuItem.addActionListener(menuItemListener);
        pasteMenuItem.addActionListener(menuItemListener);

        final CheckboxMenuItem showWindowMenu = new CheckboxMenuItem("Show About",
true);
        showWindowMenu.addItemListener(new ItemListener() {
            public void itemStateChanged(ItemEvent e) {
                if (showWindowMenu.getState()) {
                    menuBar.add(aboutMenu);
                } else {
                    menuBar.remove(aboutMenu);
                }
            }
        });

        // add menu items to menus
        fileMenu.add(newMenuItem);
        fileMenu.add(openMenuItem);
        fileMenu.add(saveMenuItem);
        fileMenu.addSeparator();
        fileMenu.add(showWindowMenu);
        fileMenu.addSeparator();
        fileMenu.add(exitMenuItem);

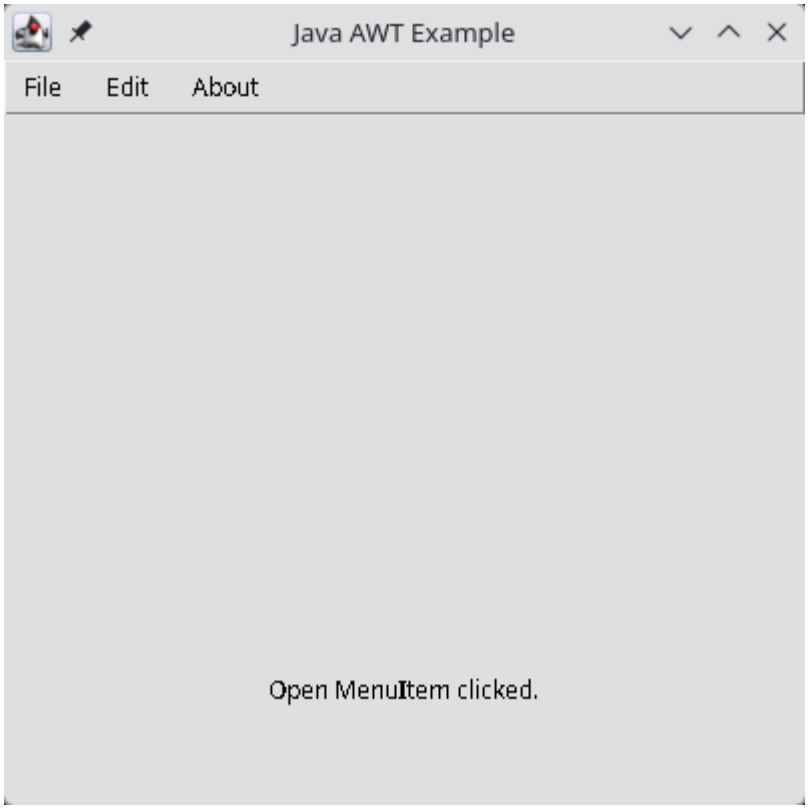
        editMenu.add(cutMenuItem);
        editMenu.add(copyMenuItem);
        editMenu.add(pasteMenuItem);

        // add menu to menubar
        menuBar.add(fileMenu);
        menuBar.add(editMenu);
        menuBar.add(aboutMenu);

        // add menubar to the frame
        mainFrame.setMenuBar(menuBar);
        mainFrame.setVisible(true);
    }

    class MenuItemListener implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            statusLabel.setText(e.getActionCommand() + " MenuItem clicked.");
        }
    }
}
```

Output:


Java AWT Example
File
Edit
About
Open MenuItem clicked.

Practical No. 11

Aim: Write a SWING based java application to implement different Layout Manager.

Algorithm:

Step 1: Start.

Step 2: Create class LayoutManagerTest which extends JFrame and set BorderLayout on the frame.

Step 3: Create a panel with FlowLayout and add buttons One, Two and Three (North).

Step 4: Create a panel with FlowLayout and add the label "This is South" (South).

Step 5: Create a panel with GridLayout(1, 3) for labels and a panel with GridLayout(3, 1) for buttons Four, Five and Six.

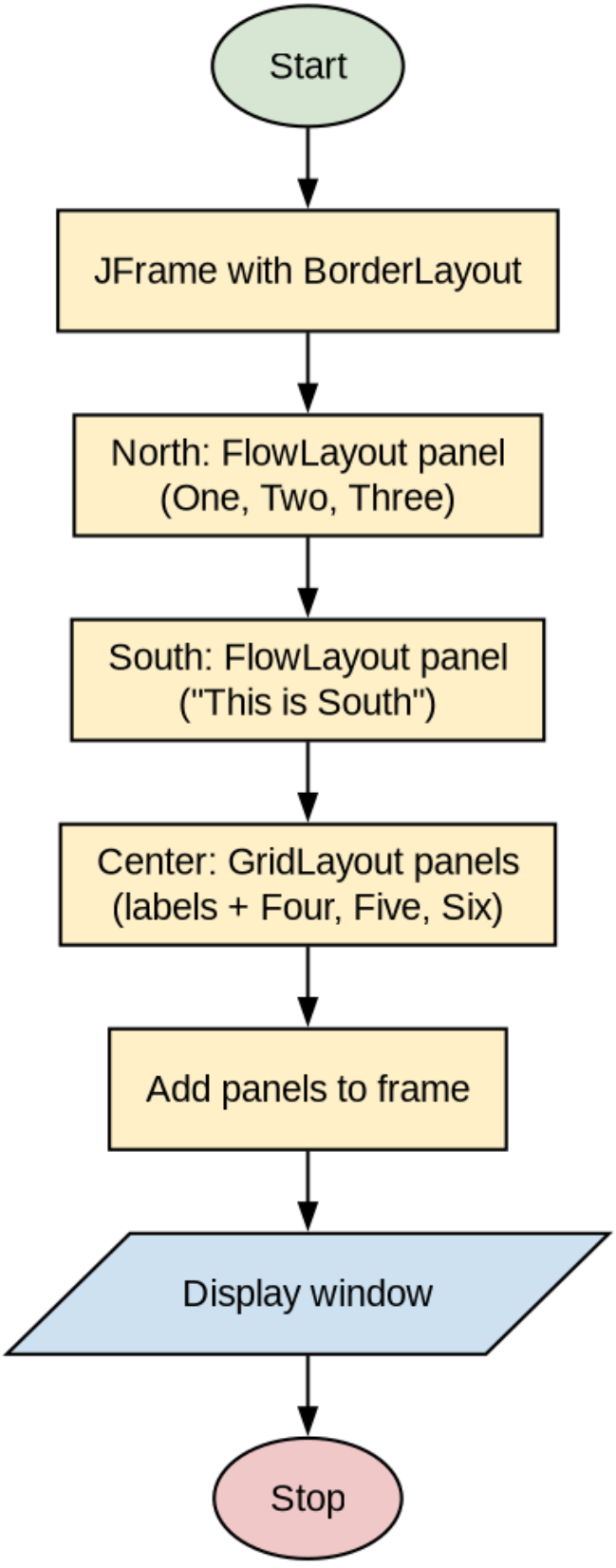
Step 6: Put both grid panels into a GridLayout(2, 1) panel (Center).

Step 7: Add the panels to the frame at NORTH, SOUTH and CENTER and make it visible.

Step 8: Stop.

Note: *The output was not given in the answer sheet; it is added here. Please test and run it yourself before writing.*

Flowchart:


Start
JFrame with BorderLayout
North: FlowLayout panel
(One, Two, Three)
South: FlowLayout panel
("This is South")
Center: GridLayout panels
(labels + Four, Five, Six)
Add panels to frame
Display window
Stop

Program:

```
import java.awt.*;
import javax.swing.*;

public class LayoutManagerTest extends JFrame
{
    JPanel flowLayoutPanel1, flowLayoutPanel2, gridLayoutPanel1,
           gridLayoutPanel2, gridLayoutPanel3;
    JButton one, two, three, four, five, six;
    JLabel bottom, lbl1, lbl2, lbl3;

    public LayoutManagerTest()
    {
        setTitle("Layout Manager Test");
        setLayout(new BorderLayout());

        // FlowLayout - North
        flowLayoutPanel1 = new JPanel();
        one = new JButton("One");
        two = new JButton("Two");
        three = new JButton("Three");
        flowLayoutPanel1.setLayout(new FlowLayout(FlowLayout.CENTER));
        flowLayoutPanel1.add(one);
        flowLayoutPanel1.add(two);
        flowLayoutPanel1.add(three);

        // FlowLayout - South
        flowLayoutPanel2 = new JPanel();
        bottom = new JLabel("This is South");
        flowLayoutPanel2.setLayout(new FlowLayout(FlowLayout.CENTER));
        flowLayoutPanel2.add(bottom);

        // GridLayout - Center
        gridLayoutPanel1 = new JPanel();
        gridLayoutPanel2 = new JPanel();
        gridLayoutPanel3 = new JPanel();

        lbl1 = new JLabel("One");
        lbl2 = new JLabel("Two");
        lbl3 = new JLabel("Three");

        four = new JButton("Four");
        five = new JButton("Five");
        six = new JButton("Six");

        gridLayoutPanel2.setLayout(new GridLayout(1, 3, 3, 5));
        gridLayoutPanel2.add(lbl1);
        gridLayoutPanel2.add(lbl2);
        gridLayoutPanel2.add(lbl3);

        gridLayoutPanel3.setLayout(new GridLayout(3, 1, 5, 5));
        gridLayoutPanel3.add(four);
        gridLayoutPanel3.add(five);
        gridLayoutPanel3.add(six);

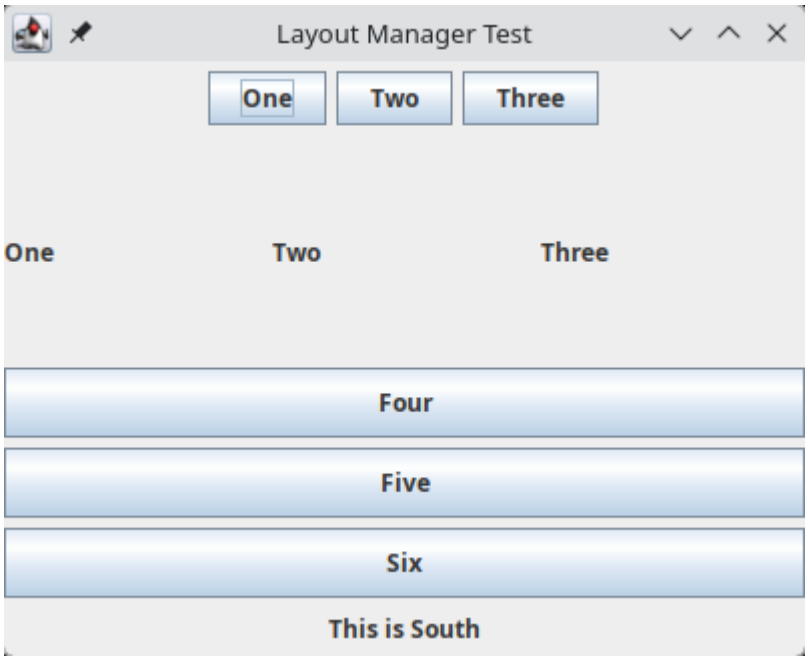
        gridLayoutPanel1.setLayout(new GridLayout(2, 1));
        gridLayoutPanel1.add(gridLayoutPanel2);
        gridLayoutPanel1.add(gridLayoutPanel3);

        // BorderLayout
        add(flowLayoutPanel1, BorderLayout.NORTH);
        add(flowLayoutPanel2, BorderLayout.SOUTH);
        add(gridLayoutPanel1, BorderLayout.CENTER);
```

```
        setSize(400, 325);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public static void main(String[] args)
    {
        new LayoutManagerTest();
    }
}
```

Output:



Practical No. 12

Aim: Write a java application to create server and client program using TCP/IP protocol.

Algorithm:

Step 1: Start.

Step 2: Server: create ServerSocket on port 420 and wait for a client using accept().

Step 3: Client: create Socket to "localhost" on port 420.

Step 4: Client: convert the string "good morning mca" to bytes and write it to the socket's OutputStream.

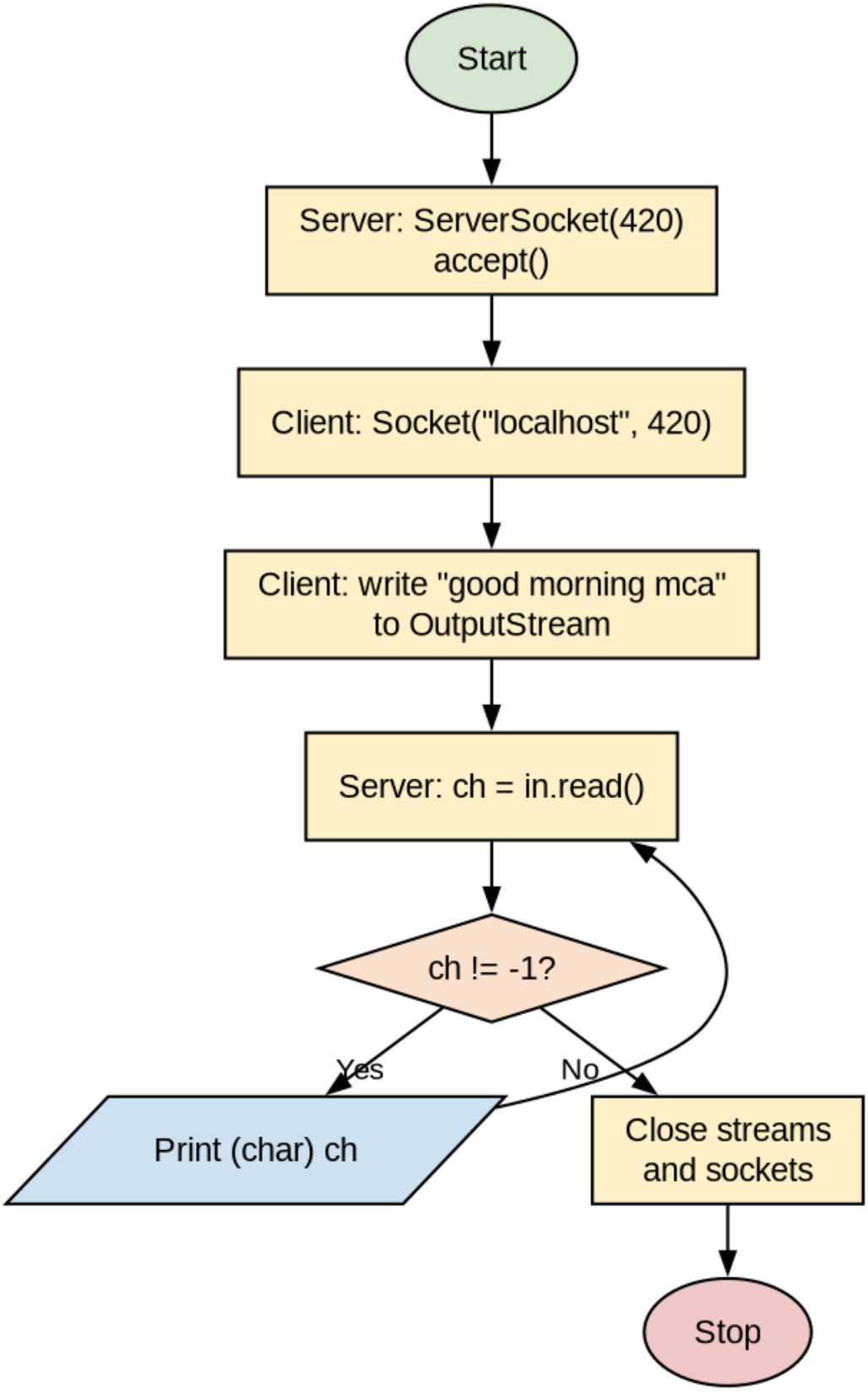
Step 5: Server: read characters from the InputStream until -1 and print them.

Step 6: Close the streams and sockets.

Step 7: Stop.

Note: *The answer is corrected here. The client connects to "localhost" (same computer) instead of 192.168.6.9. The server output is corrected to "good morning mca" (the answer sheet showed "good morning mca-msc", which the client never sends). A message line is added to the client. Please test and run it yourself before writing.*

Flowchart:


Start
Server: ServerSocket(420)
accept()
Client: Socket("localhost", 420)
Client: write "good morning mca"
to OutputStream
Server: ch = in.read()
ch != -1?
Yes
No
Print (char) ch
Close streams
and sockets
Stop

Program:

```
File: MyServer.java
-------------------
import java.net.*;
import java.io.*;

class MyServer
{
    public static void main(String args[])
    {
        try
        {
            ServerSocket s1 = new ServerSocket(420);
            Socket s2 = s1.accept();
            InputStream in = s2.getInputStream();
            int ch;
            while ((ch = in.read()) != -1)
            {
                System.out.print((char) ch);
            }
            in.close();
            s2.close();
            s1.close();
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
    }
}

File: MyClient1.java
--------------------
import java.net.*;
import java.io.*;

class MyClient1
{
    public static void main(String args[])
    {
        try
        {
            Socket s = new Socket("localhost", 420);
            OutputStream out = s.getOutputStream();
            String str = "good morning mca";
            byte b[] = str.getBytes();
            out.write(b);
            out.close();
            s.close();
            System.out.println("Message sent to server: " + str);
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

```
Server (Command Prompt 1):
F:\j-pro>javac MyServer.java
F:\j-pro>java MyServer
good morning mca

Client (Command Prompt 2):
F:\j-pro>javac MyClient1.java
F:\j-pro>java MyClient1
Message sent to server: good morning mca
```

Practical No. 13

Aim: Write a java application to create server and client program using Datagram.

Algorithm:

Step 1: Start.

Step 2: Receiver: create DatagramSocket on port 102 and an empty DatagramPacket, then wait using receive().

Step 3: Sender: create DatagramSocket on port 101.

Step 4: Sender: convert "good morning" to bytes and create a DatagramPacket addressed to localhost, port 102.

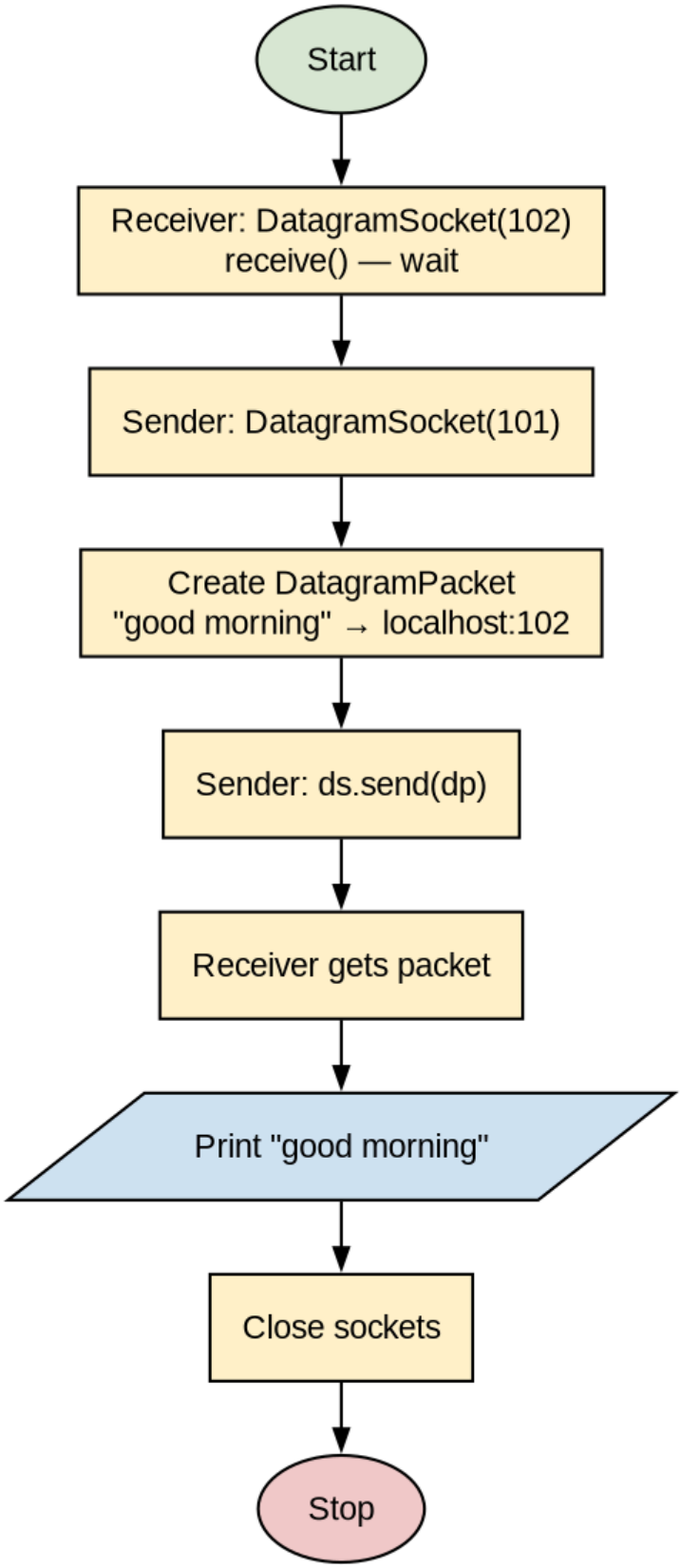
Step 5: Sender: send the packet using ds.send() and close the socket.

Step 6: Receiver: receive the packet, convert the bytes to a String and print it.

Step 7: Close the sockets. (Run Receiver first, then Sender.)

Step 8: Stop.

Flowchart:


Start
Receiver: DatagramSocket(102)
receive() — wait
Sender: DatagramSocket(101)
Create DatagramPacket
"good morning" → localhost:102
Sender: ds.send(dp)
Receiver gets packet
Print "good morning"
Close sockets
Stop

Program:

```
File: Sender.java
-----------------
import java.net.*;

public class Sender {
    public static void main(String args[]) {
        try {
            DatagramSocket ds = new DatagramSocket(101);
            String str = "good morning";
            byte b[] = str.getBytes();

            DatagramPacket dp = new DatagramPacket(
                    b,
                    b.length,
                    InetAddress.getByName("localhost"),
                    102
            );
            ds.send(dp);
            ds.close();
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}

File: Receiver.java
-------------------
import java.net.*;

public class Receiver {
    public static void main(String args[]) {
        try {
            DatagramSocket ds = new DatagramSocket(102);
            byte b[] = new byte[100];

            DatagramPacket dp = new DatagramPacket(b, b.length);
            ds.receive(dp);
            System.out.println(new String(b).trim());

            ds.close();
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:

```
Receiver (Command Prompt 1 - run first):
F:\j-pro>javac Receiver.java
F:\j-pro>java Receiver
good morning

Sender (Command Prompt 2):
F:\j-pro>javac Sender.java
F:\j-pro>java Sender
```

Practical No. 14

Aim: Write a java application to implement RMI program for addition of two numbers.

Algorithm:

Step 1: Start.

Step 2: Create remote interface Adder which extends Remote and declares add(int x, int y) throws RemoteException.

Step 3: Create class AdderRemote which extends UnicastRemoteObject, implements Adder and returns x + y.

Step 4: Create MyServer which creates an AdderRemote object and binds it using Naming.rebind("hello", stub).

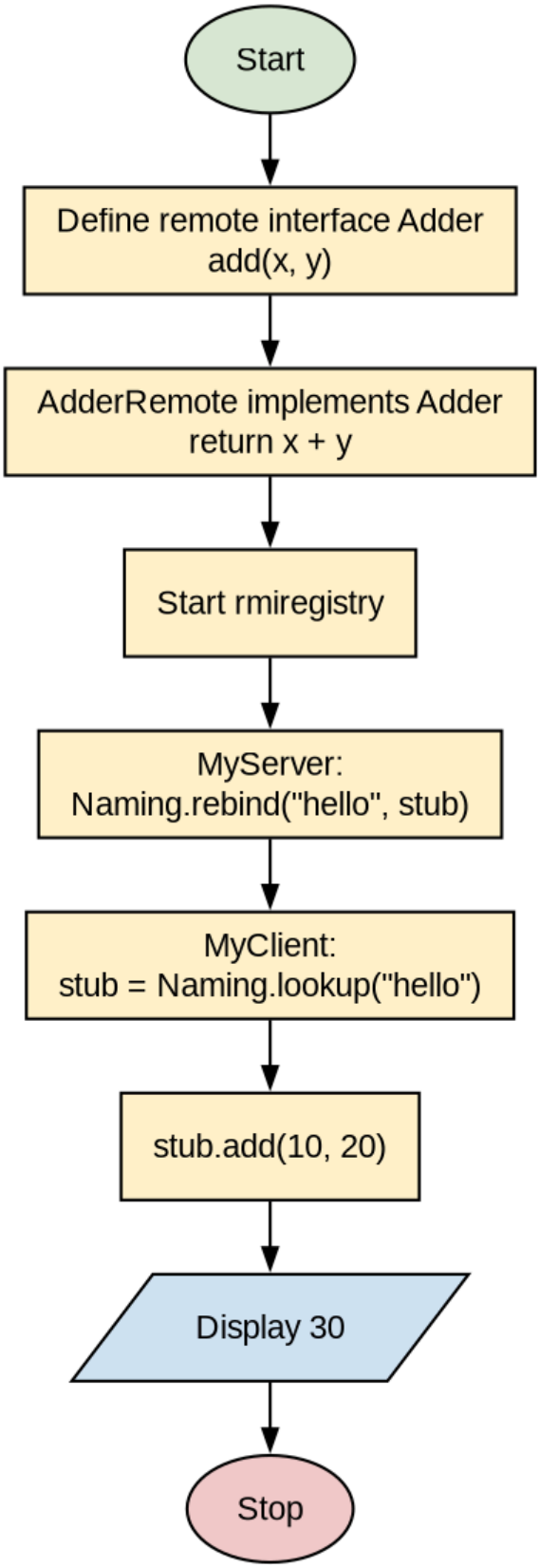
Step 5: Create MyClient which gets the remote object using Naming.lookup("hello") and calls stub.add(10, 20).

Step 6: Compile all files, start rmiregistry, run MyServer, then run MyClient.

Step 7: Display the result returned by the server.

Step 8: Stop.

Flowchart:


Start
Define remote interface Adder
add(x, y)
AdderRemote implements Adder
return x + y
Start rmiregistry
MyServer:
Naming.rebind("hello", stub)
MyClient:
stub = Naming.lookup("hello")
stub.add(10, 20)
Display 30
Stop

Program:

```
File: Adder.java
----------------
import java.rmi.*;

public interface Adder extends Remote {
    public int add(int x, int y) throws RemoteException;
}

File: AdderRemote.java
----------------------
import java.rmi.*;
import java.rmi.server.*;

public class AdderRemote extends UnicastRemoteObject implements Adder
{
    AdderRemote() throws RemoteException
    {
        super();
    }

    public int add(int x, int y)
    {
        return x + y;
    }
}

File: MyServer.java
-------------------
import java.rmi.*;
import java.rmi.registry.*;

public class MyServer {
    public static void main(String[] args) {
        try
        {
            Adder stub = new AdderRemote();
            Naming.rebind("hello", stub);
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}

File: MyClient.java
-------------------
import java.rmi.*;

public class MyClient {
    public static void main(String[] args) {
        try {
            Adder stub = (Adder) Naming.lookup("hello");
            System.out.println(stub.add(10, 20));
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:

```
Command Prompt 1:
F:\rmi>javac *.java
F:\rmi>start rmiregistry
F:\rmi>java MyServer

Command Prompt 2:
F:\rmi>java MyClient
30
```

Practical No. 15

Aim: Write a java application to insert data into database using JDBC.

Pre-requisite: Create the database and table in MySQL (see below) and keep mysql-connector-j.jar in the classpath.

Algorithm:

Step 1: Start.

Step 2: Create the database college and table sdata(name, age, address) in MySQL.

Step 3: Create a JFrame with labels and text fields for Name, Age and Address, and buttons Save and Clear.

Step 4: Register an ActionListener on both buttons.

Step 5: When Save is clicked, read the values from the text fields.

Step 6: Load the driver using Class.forName("com.mysql.cj.jdbc.Driver") and connect using DriverManager.getConnection().

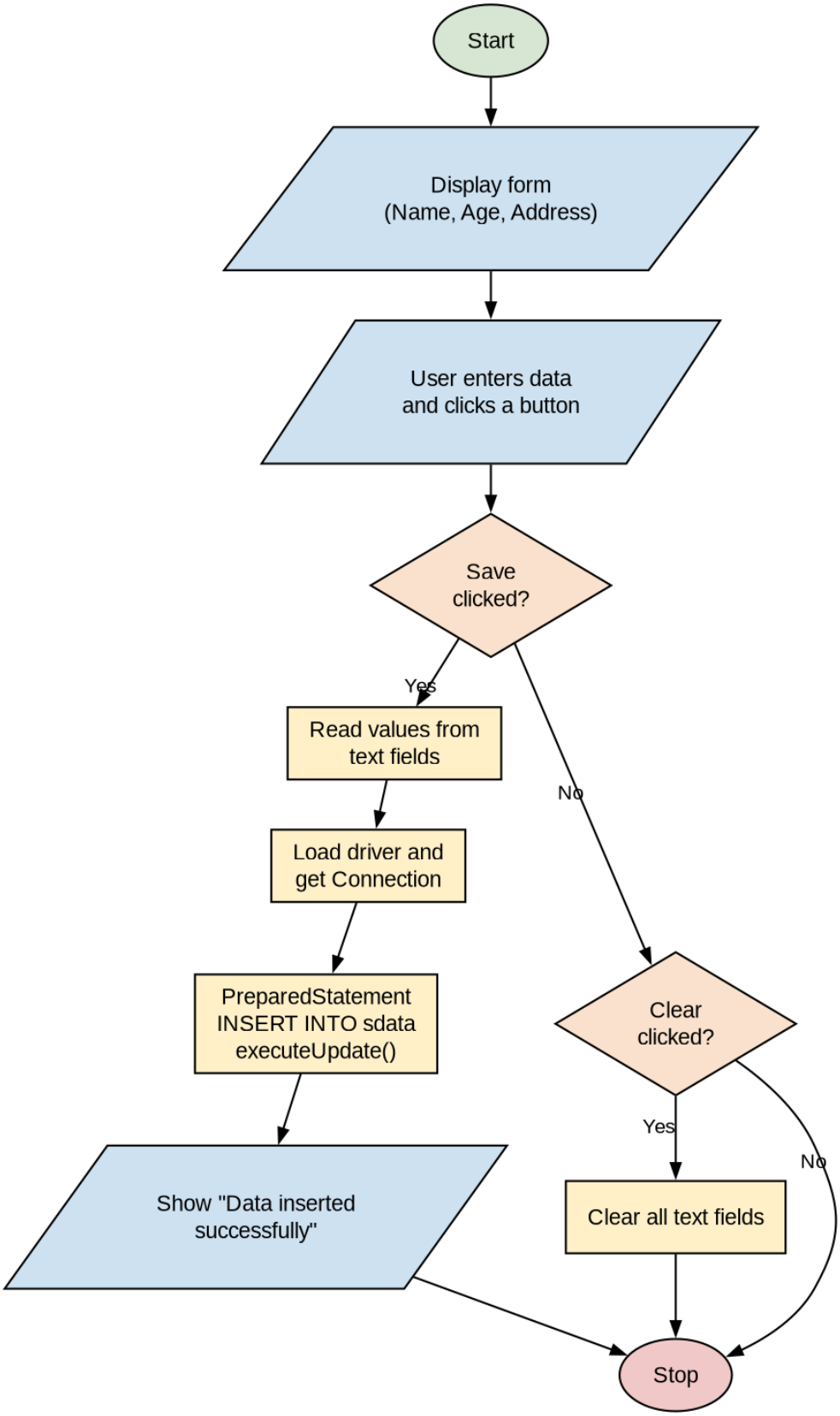
Step 7: Insert the values using PreparedStatement and executeUpdate(), then show "Data inserted successfully".

Step 8: When Clear is clicked, clear all text fields.

Step 9: Stop.

Note: *The answer is corrected here. The JDBC-ODBC bridge (sun.jdbc.odbc.JdbcOdbcDriver) has been removed from Java 8 onwards, so the program now uses the MySQL JDBC driver. The Clear button is also made to work. The output was not given in the answer sheet; it is added here. Please test and run it yourself before writing.*

Flowchart:


Start
Display form
(Name, Age, Address)
User enters data
and clicks a button
Save
clicked?
Yes
No
Read values from
text fields
Load driver and
get Connection
PreparedStatement
INSERT INTO sdata
executeUpdate()
Show "Data inserted
successfully"
Clear
clicked?
Yes
No
Clear all text fields
Stop

Program:

```
File: sdata.sql
---------------
CREATE DATABASE college;
USE college;
CREATE TABLE sdata (
    name    VARCHAR(30),
    age     INT,
    address VARCHAR(50)
);

File: FormData1.java
--------------------
import java.awt.*;
import java.sql.*;
import javax.swing.*;
import java.awt.event.*;

class FormData1 extends JFrame implements ActionListener {
    JLabel l1, l2, l3;
    JTextField t1, t2, t3;
    JButton b1, b2;

    public FormData1() {
        setTitle("Student Form");
        l1 = new JLabel("Enter Name: ");
        l2 = new JLabel("Enter age: ");
        l3 = new JLabel("Address: ");

        t1 = new JTextField(20);
        t2 = new JTextField(20);
        t3 = new JTextField(20);

        b1 = new JButton("Save");
        b2 = new JButton("Clear");

        b1.addActionListener(this);
        b2.addActionListener(this);

        JPanel p = new JPanel(new GridLayout(4, 2));
        p.add(l1);
        p.add(t1);
        p.add(l2);
        p.add(t2);
        p.add(l3);
        p.add(t3);
        p.add(b1);
        p.add(b2);

        add(p);
        setSize(400, 400);
        setVisible(true);
        pack();

        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });
    }

    public static void main(String[] args) {
        new FormData1();
```

```
    }

    public void actionPerformed(ActionEvent e) {

        if (e.getSource() == b1) {
            String v1 = t1.getText();
            int v2 = Integer.parseInt(t2.getText());
            String v3 = t3.getText();

            try {
                // MySQL JDBC driver (JDBC-ODBC bridge is removed from Java 8
onwards)
                Class.forName("com.mysql.cj.jdbc.Driver");
                Connection con = DriverManager.getConnection(
                        "jdbc:mysql://localhost:3306/college", "root", "");

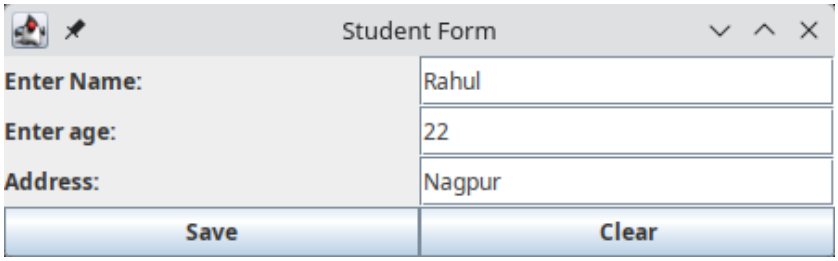
                PreparedStatement ps = con.prepareStatement("INSERT INTO sdata
VALUES(?, ?, ?)");
                ps.setString(1, v1);
                ps.setInt(2, v2);
                ps.setString(3, v3);

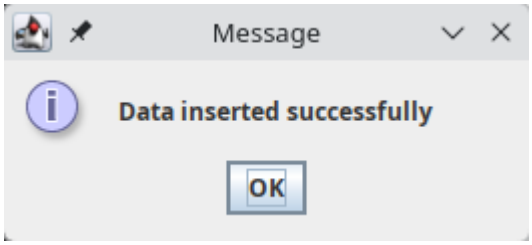
                ps.executeUpdate();
                JOptionPane.showMessageDialog(null, "Data inserted successfully");
                con.close();
            }
            catch (Exception ex) {
                System.out.println(ex);
            }
        }
        if (e.getSource() == b2) {
            t1.setText("");
            t2.setText("");
            t3.setText("");
        }
    }
}
```

Output:

```
C:\jdbc>javac FormData1.java
C:\jdbc>java -cp .;mysql-connector-j.jar FormData1

mysql> SELECT * FROM sdata;
+-------+------+---------+
| name  | age  | address |
+-------+------+---------+
| Rahul |   22 | Nagpur  |
+-------+------+---------+
1 row in set
```




Message
Data inserted successfully
OK